Python E-Course

Module 10 — Modules & Packages

Detailed Notes

Module Overview

Level: Intermediate

Duration: ~1 week

Prerequisites: Modules 1-9

What You'll Learn

- Import modules and use their functions.
- Create your own modules and packages.
- Manage dependencies with pip and virtual environments.
- Structure a multi-file Python project.

Lessons

#	Lesson	Duration
1	The import System	30 min
2	Creating Your Own Modules	30 min
3	Packages and __init__.py	30 min
4	pip and Virtual Environments	35 min
5	Project Structure & if __name__	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 11: Functional Programming.

Lesson 1: The import System

Module: 10 — Modules & Packages

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Use import, from ... import, as, and * syntaxes.
- Use standard library modules: math, random, datetime, os, sys.
- Avoid common import anti-patterns.

Prerequisites

- Modules 1-9

1. Why This Matters

You've already imported a few modules (csv, json, pathlib, re). The standard library is enormous — hundreds of modules covering math, dates, networking, threading, and more. Knowing how to use them saves you from reinventing wheels.

2. Concept

2.1 Plain import

```
import math
print(math.sqrt(16))     # 4.0
print(math.pi)
```

Access names via the module's namespace.

2.2 from ... import

```
from math import sqrt, pi
print(sqrt(16))
print(pi)
```

Names go directly into your namespace. Use sparingly — too many from x import y lines hide where things come from.

2.3 Aliasing with as

```
import datetime as dt
import numpy as np         # common convention (3rd party)
```

Useful for long module names or to avoid name collisions.

2.4 The * import (avoid)

```
from math import *
```

Imports everything. Don't — pollutes namespace and you lose track of where names come from.

2.5 The standard library tour

Some modules you'll meet:

Module	What it does
math	math functions, constants
random	random numbers, sampling
datetime	dates, times, deltas
os / pathlib	filesystem
sys	interpreter info, argv, exit
json	JSON encoding/decoding
csv	CSV files
re	regex
collections	Counter, defaultdict, deque
itertools	iterator tools
functools	reduce, lru_cache, partial
statistics	mean, median, stdev
urllib.request	basic HTTP
subprocess	run other programs
argparse	command-line argument parsing

2.6 How Python finds modules

It searches sys.path — your current directory, then standard library locations, then site-packages (where pip installs to). You can inspect:

```
import sys
print(sys.path)
```

2.7 Selective imports help readability

```
# Less clear
import datetime
now = datetime.datetime.now()

# Clearer
from datetime import datetime
now = datetime.now()
```

3. Code Examples

Example 1: math

```
import math
print(math.sqrt(2))
print(math.factorial(5))
print(math.floor(3.7), math.ceil(3.2))
print(math.pi)
```

Expected Output:

```
1.4142135623730951
120
```

```
3 4
3.141592653589793
```

Example 2: random

```
import random
random.seed(42)
print(random.randint(1, 10))
print(random.choice(["apple", "banana", "cherry"]))
print(random.sample(range(100), 3))
```

Expected Output:

```
2
banana
[81, 14, 3]
```

Explanation: seed(42) makes the run reproducible.

Example 3: datetime

```
from datetime import datetime, timedelta
now = datetime.now()
print(now.strftime("%Y-%m-%d %H:%M"))
later = now + timedelta(days=7)
print(later.date())
```

Expected Output (example, varies by clock):

```
2026-05-23 11:30
2026-05-30
```

Example 4: collections.Counter

```
from collections import Counter
text = "the rain in spain stays mainly in the plain"
print(Counter(text.split()).most_common(3))
```

Expected Output: [('the', 2), ('in', 2), ('rain', 1)]

Example 5: alias

```
import json
data = {"a": 1, "b": [1, 2, 3]}
print(json.dumps(data, indent=2))
```

Expected Output:

```
{
  "a": 1,
  "b": [
    1,
    2,
    3
  ]
}
```

4. Common Mistakes

Mistake	What Happens	Fix
from math import *	Namespace pollution	Import what you use
Importing inside a hot loop	Slight performance cost	Import at top of file
Circular imports	ImportError	Restructure modules
Naming your file random.py	Shadows stdlib!	Don't reuse stdlib module names

5. Practice Tasks

- Use random.choice to pick a fortune from a list of 5 strings.
- Use datetime to print today's date in DD-MM-YYYY.
- Use collections.Counter to count letters in "abracadabra".

6. Key Takeaways

- import X brings in the module; from X import y imports specific names.
- Use as for aliasing; avoid from X import *.
- The standard library is huge — check before installing third-party packages.
- Don't shadow stdlib names with your filenames.

7. What's Next

Create your own modules — split your code across files.

Lesson 2: Creating Your Own Modules

Module: 10 — Modules & Packages

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Create a Python module (a .py file) and import it from another.
- Use __name__ to detect script vs imported execution.
- Use __all__ to define a module's public API.

Prerequisites

- Module 10 Lesson 1

1. Why This Matters

Once a program grows past ~200 lines, you'll want to split it into multiple files. Each .py file is a module. Knowing how to organize them — and what happens at import time — keeps your project navigable.

2. Concept

2.1 Any .py file is a module

Create mathx.py:

```
def double(x):
    return x * 2

def triple(x):
    return x * 3
```

Then main.py in the same folder:

```
import mathx
print(mathx.double(5))
```

That's the whole concept.

2.2 Importing names

```
from mathx import double
print(double(5))
```

2.3 What happens at import

Python:

- Finds the file on sys.path.
- Executes it top to bottom (only once per Python session).

- Stores its names in sys.modules and binds the module object to the name you used.

If you re-import, Python uses the cached module — no re-execution.

2.4 __name__

Every module has a built-in name. When run directly, it's "__main__". When imported, it's the module's actual name.

```
# mathx.py
def double(x):
    return x * 2

if __name__ == "__main__":
    print("Running directly")
    print(double(5))
```

This pattern lets a file be both a library and a script.

2.5 __all__ — define your public API

```
# mathx.py
__all__ = ["double", "triple"]

def double(x): return x * 2
def triple(x): return x * 3
def _helper(): ...   # not exported
```

from mathx import will only import names in __all__. (You should still avoid import — but __all__ documents intent.)

2.6 Module docstring

The first triple-quoted string in a module is its docstring:

```
"""Math helpers used across the app."""

def double(x): ...
```

3. Code Examples

Example 1: Simple module

Save mathx.py:

```
def double(x):
    return x * 2

def triple(x):
    return x * 3
```

Then main.py:

```
import mathx
print(mathx.double(7))
print(mathx.triple(7))
```

Expected Output:

```
14
21
```

Example 2: __name__ guard

util.py:

```
def greet(name):
    return f"Hi, {name}"

if __name__ == "__main__":
    print(greet("World"))
```

Running python util.py:

Expected Output: Hi, World

Running python -c "import util":

Expected Output: (nothing — the guard skipped the print)

Example 3: from import with alias

numbers.py:

```
def sq(x):
    return x * x
```

main.py:

```
from numbers_helper import sq as square    # if you named it numbers_helper to
avoid stdlib clash
print(square(5))
```

Expected Output: 25

(Note: stdlib has a numbers module — pick a different filename to avoid collision.)

Example 4: A small "library"

stringx.py:

```
"""Small string helpers."""

__all__ = ["snake_to_camel", "trim_each_line"]

def snake_to_camel(s):
    parts = s.split("_")
    return parts[0] + "".join(p.title() for p in parts[1:])
```

```
def trim_each_line(text):
    return "\n".join(line.strip() for line in text.splitlines())
```

main.py:

```
from stringx import snake_to_camel, trim_each_line
print(snake_to_camel("my_long_name"))
print(trim_each_line("  hi  \n  there  "))
```

Expected Output:

```
myLongName
hi
there
```

4. Common Mistakes

Mistake	What Happens	Fix
Naming file the same as a stdlib module	Shadows stdlib	Pick another name
Side effects (prints, network) at top level of imported module	Run every import	Move into if __name__ == "__main__":
Mutual imports	ImportError	Restructure or use lazy import inside function
Expecting changes to a module file to live-reload	Cached after first import	Restart Python (or importlib.reload, but uncommon)

5. Practice Tasks

- Create mathx.py with double and square. Import and use from another file.
- Add if __name__ == "__main__": self-test to that module.
- Add __all__ = ["double"] and confirm from mathx import * only exposes double.

6. Key Takeaways

- A .py file is a module.
- if __name__ == "__main__": makes a file work as both script and library.
- __all__ documents the public API.
- Don't shadow stdlib module names.

7. What's Next

When several modules belong together, group them into a package.

Lesson 3: Packages and __init__.py

Module: 10 — Modules & Packages

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Build a package by grouping modules in a folder.
- Use __init__.py to expose package-level names.
- Use relative imports inside a package.

Prerequisites

- Module 10 Lesson 2

1. Why This Matters

A module is one file. A package is a folder of related modules. Real projects ship as packages — requests, pandas, numpy are all packages. Knowing how to structure one means your project can grow without becoming chaos.

2. Concept

2.1 Minimal package

```
mylib/
├── __init__.py
├── core.py
└── utils.py
```

The folder mylib/ is a package because it has __init__.py (the file can be empty).

2.2 Importing from a package

```
import mylib.core
from mylib.utils import helper
```

2.3 What __init__.py does

Runs when the package is first imported. Use it to:

- Re-export key names so callers can do from mylib import X:

``python # mylib/__init__.py from .core import main_function from .utils import helper ``

- Set __version__, __all__, package-level constants.

You can also leave it empty — that's fine for many small packages.

2.4 Relative vs absolute imports

Inside a package, import siblings with a leading dot:

```
# mylib/core.py
from .utils import helper        # relative — same package
from mylib.utils import helper   # absolute — works if mylib is on sys.path
```

Absolute imports are usually clearer; relative imports keep the package self-contained when its name changes.

2.5 Subpackages

Nest packages by nesting folders, each with __init__.py:

```
mylib/
├── __init__.py
├── core.py
├── parsers/
│   ├── __init__.py
│   ├── json_parser.py
│   └── xml_parser.py
└── utils/
    ├── __init__.py
    └── strings.py
```

Import: from mylib.parsers.json_parser import parse.

2.6 Namespace packages (advanced)

Since Python 3.3, you don't strictly need __init__.py. But for clarity and tooling support, always include __init__.py for now.

3. Code Examples

Example 1: Tiny package

Create a folder greetings/ with:

greetings/__init__.py:

```
"""Greeting helpers."""

from .english import hello
from .french import bonjour
```

greetings/english.py:

```
def hello(name):
    return f"Hello, {name}!"
```

greetings/french.py:

```
def bonjour(name):
    return f"Bonjour, {name}!"
```

Use it from main.py:

```
from greetings import hello, bonjour
print(hello("Alex"))
print(bonjour("Alex"))
```

Expected Output:

```
Hello, Alex!
Bonjour, Alex!
```

Example 2: Subpackage

```
shop/
├── __init__.py
├── core.py
└── models/
    ├── __init__.py
    └── product.py
```

shop/models/product.py:

```
from dataclasses import dataclass

@dataclass
class Product:
    name: str
    price: float
```

shop/core.py:

```
from .models.product import Product

def make_demo():
    return [Product("apple", 50), Product("bread", 40)]
```

Use:

```
from shop.core import make_demo
for p in make_demo():
    print(p)
```

Expected Output:

```
Product(name='apple', price=50)
Product(name='bread', price=40)
```

Example 3: Empty __init__.py

Just touch greetings/__init__.py (zero bytes). You can then from greetings.english import hello — empty __init__.py is enough to declare the package.

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting __init__.py	Newer Python tolerates it (namespace pkg) but tooling	Always include it

	may not	
Using from . outside a package	ImportError: attempted relative import with no known parent package	Only use relative imports inside an actual package
Importing a module that imports the same parent	Circular import	Refactor or import inside the function
Adding heavy work in __init__.py	Slows every import	Keep __init__.py small

5. Practice Tasks

- Create a mylib/ package with two modules (a.py, b.py). Import a function from each in a script.
- Add re-exports in mylib/__init__.py so from mylib import func_a, func_b works.
- Add a subpackage mylib/utils/ with one module and import from it.

6. Key Takeaways

- A folder + __init__.py = a package.
- __init__.py runs on first import — use it for re-exports and package metadata.
- Subpackages let you nest deeper structure.
- Always include __init__.py for clarity.

7. What's Next

pip and virtual environments — install third-party packages without messing up other projects.

Lesson 4: pip and Virtual Environments

Module: 10 — Modules & Packages

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Install and uninstall third-party packages with pip.
- Create and activate a virtual environment with venv.
- Pin dependencies with requirements.txt.

Prerequisites

- Module 10 Lesson 3

1. Why This Matters

The Python ecosystem has ~500,000 packages on PyPI. pip installs them; virtual environments keep each project's dependencies isolated so installing requests==2.31.0 for one project doesn't break another that needs requests==2.10.0. This is the single most important "professional Python" habit.

2. Concept

2.1 pip basics

```
pip install requests           # latest
pip install requests==2.31.0   # pin version
pip install -U requests        # upgrade
pip uninstall requests
pip list                       # installed packages
pip show requests              # details
```

On some systems, use pip3 or python -m pip.

2.2 Why virtual environments

Installing globally puts all packages in one place — version conflicts are inevitable. A virtual environment is a folder containing its own Python and its own site-packages. Active one venv = isolated install space.

2.3 Creating and activating venv

```
python -m venv .venv
```

This creates a .venv/ folder. Activate it:

Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

Windows (cmd):

```
.venv\Scripts\activate.bat
```

macOS/Linux:

```
source .venv/bin/activate
```

Once active, your shell prompt shows (.venv). Now pip install only affects this venv.

Deactivate with deactivate.

2.4 requirements.txt

Standard way to record dependencies:

```
# requirements.txt
requests==2.31.0
python-dateutil>=2.8
```

Install all:

```
pip install -r requirements.txt
```

Freeze your current environment:

```
pip freeze > requirements.txt
```

pip freeze outputs exact versions of everything currently installed (including transitive dependencies). For application code, that's fine. For libraries, prefer to list only your direct deps.

2.5 The .venv should NOT be committed

Add to .gitignore:

```
.venv/
__pycache__/
*.pyc
```

Recreate on each machine via pip install -r requirements.txt.

2.6 Alternatives

- pipx — installs CLI tools globally but isolated.
- poetry, uv, pipenv, hatch — modern packaging/dep managers. After this course, look into uv or poetry.

For now, venv + pip is the canonical foundation.

3. Code Examples

Example 1: Setup

In your terminal (don't type the $ — that's a prompt symbol):

```
$ python -m venv .venv
$ source .venv/bin/activate     # mac/linux
(.venv) $ pip install requests
(.venv) $ pip list
```

Expected Output (partial):

```
Package    Version
---------- -------
certifi    2024.7.4
charset-normalizer 3.3.2
idna       3.7
pip        23.2.1
requests   2.31.0
urllib3    2.2.1
```

Example 2: A script using a third-party package

With the venv active and requests installed:

```
import requests
resp = requests.get("https://httpbin.org/json")
print(resp.status_code)
print(resp.json()["slideshow"]["title"])
```

Expected Output:

```
200
Sample Slide Show
```

Example 3: Freeze and reinstall

```
(.venv) $ pip freeze > requirements.txt
(.venv) $ cat requirements.txt
certifi==2024.7.4
charset-normalizer==3.3.2
idna==3.7
requests==2.31.0
urllib3==2.2.1
```

Recreate in another folder:

```
$ python -m venv .venv
$ source .venv/bin/activate
(.venv) $ pip install -r requirements.txt
```

Example 4: Specific version

```
pip install "Django>=4.2,<5"
```

Quoted to prevent the shell from interpreting </>.

4. Common Mistakes

Mistake	What Happens	Fix
Installing without a venv	Polluted global Python	Always venv per project

Forgetting to activate	Installing to system Python	Confirm (.venv) in prompt
Committing .venv/ to git	Huge repo, OS-specific	.gitignore it
Using sudo pip install on macOS/Linux	Breaks system Python	Use venv
Mixing pip and pip3	Different Pythons, different installs	Use python -m pip to be safe

5. Practice Tasks

- Create a venv called .venv in a new folder. Activate it. Run which python (or where python) to confirm.
- Install requests. Use it to GET https://httpbin.org/get and print the status code.
- Run pip freeze > requirements.txt. Open the file and inspect.

6. Key Takeaways

- pip install pkg installs a package; venv isolates that install per-project.
- Always activate your venv before installing or running.
- Commit requirements.txt, not .venv/.

7. What's Next

The last lesson — sensible project structure and the if __name__ == "__main__" idiom.

Lesson 5: Project Structure & if __name__

Module: 10 — Modules & Packages

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Structure a Python project with multiple files and a package.
- Use the if __name__ == "__main__": idiom correctly.
- Set up a minimal pyproject.toml and README.md.

Prerequisites

- Module 10 Lesson 4

1. Why This Matters

A clean structure makes a project easy to navigate, test, and ship. There's no single "right" layout, but conventions exist — knowing them means new developers (and future-you) can read your code without a map.

2. Concept

2.1 A small but solid layout

```
myproject/
├── README.md
├── pyproject.toml         # project metadata + dependencies
├── requirements.txt       # locked deps for app code (alternative)
├── .gitignore
├── .venv/                 # NOT committed
├── src/
│   └── myapp/             # the package
│       ├── __init__.py
│       ├── main.py
│       ├── core.py
│       └── utils.py
└── tests/
    ├── __init__.py
    └── test_core.py
```

For very small projects, the src/ and tests/ folders are optional — you can put myapp/ and tests at the top level. The structure above scales nicely.

2.2 if __name__ == "__main__":

```
def main():
    print("hello")

if __name__ == "__main__":
    main()
```

When the file is run with python myapp/main.py, __name__ is "__main__" and main() runs. When the file is imported (from myapp.main import main), __name__ is "myapp.main" and main() doesn't auto-run.

This is how a file works as both a script and a library.

2.3 A minimal pyproject.toml

```
[project]
name = "myapp"
version = "0.1.0"
description = "A small Python app"
requires-python = ">=3.10"
dependencies = [
    "requests>=2.31",
]

[build-system]
requires = ["setuptools>=61"]
build-backend = "setuptools.build_meta"
```

This is the modern standard for project metadata. Tools like pip, build, uv all read it.

2.4 A README.md skeleton

```
# MyApp

Brief description.

## Install
```

python -m venv .venv source .venv/bin/activate pip install -e .

```
## Run
```

python -m myapp.main

```
## Develop
```

pip install -r requirements-dev.txt pytest

2.5 .gitignore essentials

```
.venv/
__pycache__/
*.pyc
.pytest_cache/
```

```
.coverage
*.egg-info/
dist/
build/
```

2.6 Running as a package vs as a file

```
python myapp/main.py      # runs as script — relative imports may fail
python -m myapp.main      # runs as a module — recommended
```

python -m correctly sets up the package context so relative imports work.

3. Code Examples

Example 1: Script-or-library file

mathx.py:

```
def double(x):
    return x * 2

def main():
    for i in range(5):
        print(double(i))

if __name__ == "__main__":
    main()
```

Run directly:

```
$ python mathx.py
0
2
4
6
8
```

Import it: from mathx import double — main() does not run.

Example 2: Small package + entry point

```
myapp/
├── __init__.py
└── main.py
```

myapp/main.py:

```
from .core import process     # works because we run via python -m

def main():
    process("hello")

if __name__ == "__main__":
    main()
```

myapp/core.py:

```
def process(msg):
    print("processing:", msg)
```

Run:

```
$ python -m myapp.main
processing: hello
```

Example 3: pyproject.toml install

With pyproject.toml in your project root:

```
$ pip install -e .
```

The -e means editable — installs your package such that edits take effect immediately. Now import myapp works from anywhere in the venv.

4. Common Mistakes

Mistake	What Happens	Fix
Running python pkg/main.py instead of python -m pkg.main	Relative imports fail	Always run packaged code with -m
Big __init__.py doing heavy work	Slow imports	Keep it minimal
No .gitignore	Committing .venv etc.	Add the standard ignores
Forgetting if __name__ == "__main__"	Side effects on import	Always guard entry points

5. Practice Tasks

- Take any single-file script you wrote earlier and split it into a myapp/ package with main.py and one other module.
- Add if __name__ == "__main__": main() to your entry point.
- Write a minimal pyproject.toml for it (no deps yet).

6. Key Takeaways

- Use a myapp/ package with __init__.py for anything beyond a script.
- if __name__ == "__main__": lets one file be both script and library.
- Run as python -m myapp.main to keep imports clean.
- pyproject.toml is the modern way to declare project metadata.

7. What's Next

End of Module 10. Next: Module 11 — Functional Programming.

Module 10 — Exercises

15 exercises.

10.1 import

- (E) Use math.sqrt(81) and print result.
- (M) Use random.choice to pick a fortune from a list.
- (H) Use collections.Counter to count words in a sentence.

10.2 Own modules

- (E) Create mymath.py with add(a,b). Import from another file.
- (M) Add if __name__ == "__main__": self-test that prints add(2,3).
- (H) Add __all__ = ["add"] and a private _helper. Confirm from mymath import * hides _helper.

10.3 Packages

- (E) Create a mylib/ folder with __init__.py and core.py. Import from core in a script.
- (M) Add a re-export in __init__.py so from mylib import func works.
- (H) Add a subpackage mylib/utils/ with one module.

10.4 venv/pip

- (E) Create a venv .venv and activate it.
- (M) Install requests. Run a script that GETs https://httpbin.org/get.
- (H) pip freeze > requirements.txt. Recreate the env in another folder from it.

10.5 Structure

- (E) Add if __name__ == "__main__": to any past script.
- (M) Split a script into a package with main.py and one helper module.
- (H) Write a minimal pyproject.toml for your package.

Module 10 — Quiz

10 MCQs.

Q1

Which is preferred over from math import ? A. import math B. import math as m C. from math import sqrt, pi D. All except

Answer: D (L1)

Q2

What does if __name__ == "__main__": do? A. Always runs the block B. Runs only when the file is imported C. Runs only when the file is run directly D. Imports __main__

Answer: C (L2)

Q3

A folder is a package when it contains: A. Any .py file B. __init__.py C. __main__.py D. setup.py

Answer: B (L3)

Q4

Inside a package, what does from .utils import x do? A. Imports from PyPI B. Imports from the same package's utils module C. Imports from sys.modules 'utils' D. SyntaxError

Answer: B (L3)

Q5

What does python -m venv .venv do? A. Activates a venv B. Lists venvs C. Creates a new venv in .venv/ D. Installs venv from PyPI

Answer: C (L4)

Q6

Once a venv is active, pip install requests: A. Installs globally B. Installs into the venv C. Installs to user folder D. Fails

Answer: B (L4)

Q7

requirements.txt typically: A. Stores test results B. Lists Python source files C. Pins package versions for reproducible installs D. Contains environment variables

Answer: C (L4)

Q8

Best way to run a packaged entry point: A. python pkg/main.py B. python -m pkg.main C. ./pkg/main.py D. pkg.main()

Answer: B (L5)

Q9

You named your file random.py and import random. What happens? A. Works B. ImportError C. Your file is imported instead — surprising bugs D. Python errors with a warning

Answer: C (L1)

Q10

Modern Python project metadata lives in: A. setup.py B. pyproject.toml C. Pipfile D. package.json

Answer: B (L5)

Module 10 — Assignment: Restructure the Notes App

Estimated time: 2 hours

Combines: Module 10 + Modules 7, 8

Brief

Take the Module-7/8 Notes Manager and refactor it into a proper Python package layout with a venv, requirements, README, and a python -m entry point.

Requirements

Project layout (create exactly this):

```
notes/
├── README.md
├── pyproject.toml
├── requirements.txt        # (can be empty for now — stdlib only)
├── .gitignore
└── notesapp/
    ├── __init__.py
    ├── __main__.py         # so `python -m notesapp` works
    ├── storage.py          # load/save JSON
    ├── commands.py         # add/list/view/del/find
    ├── errors.py           # custom exceptions
    └── cli.py              # parse and dispatch commands
```

Functional requirements:

- python -m notesapp launches the interactive CLI from Module-8 project.
- All file I/O goes through storage.py.
- All custom exceptions live in errors.py.
- Each command_X(...) is its own function in commands.py.
- cli.py only handles the prompt loop and dispatch.
- __init__.py exposes a __version__ = "0.1.0".
- __main__.py calls cli.run().
- pyproject.toml lists the package with requires-python = ">=3.10".
- README.md shows install + run instructions.

Constraints

- Must run with python -m notesapp from project root.
- No external dependencies (stdlib only).
- Keep the existing functionality from M7/M8 project intact.

Submission

Zip or commit the whole notes/ folder.

Grading Rubric

Criterion	Weight
Layout matches spec	25%
python -m notesapp works	25%
Separation of concerns	25%
pyproject.toml + README + .gitignore present	15%
Style + imports	10%

Module 10 — Mini Project: Password Generator Package

Overview

Build a small package passgen that generates passwords with configurable rules, plus a CLI to invoke it. Practice packaging, argparse, and a clean module layout.

Concepts Used

- Packages + __init__.py (M10 L3)
- if __name__ == "__main__": and python -m (M10 L5)
- Standard library: random, string, argparse
- Functions (M4)

Features

- Generate a password of a given length.
- Optional character classes: lower, upper, digits, symbols.
- Optional --no-ambiguous flag to exclude lookalikes like l/I/1 and 0/O.
- Default: length 16, all classes.

Layout

```
passgen/
├── README.md
├── pyproject.toml
├── passgen/
│   ├── __init__.py
│   ├── __main__.py
│   ├── core.py
│   └── cli.py
```

Starter Code

passgen/core.py:

```
import random
import string

AMBIGUOUS = set("lI1O0")

def generate(length=16, lower=True, upper=True, digits=True, symbols=True,
             no_ambiguous=False):
    pool = ""
    if lower:   pool += string.ascii_lowercase
    if upper:   pool += string.ascii_uppercase
    if digits:  pool += string.digits
    if symbols: pool += "!@#$%^&*()-_=+[]{}<>?"
    if no_ambiguous:
        pool = "".join(c for c in pool if c not in AMBIGUOUS)
    if not pool:
        raise ValueError("at least one character class must be enabled")
    return "".join(random.choice(pool) for _ in range(length))
```

passgen/cli.py:

```
import argparse
from .core import generate

def build_parser():
    p = argparse.ArgumentParser(prog="passgen", description="Generate a random
password.")
    p.add_argument("-l", "--length", type=int, default=16)
    p.add_argument("--no-lower", action="store_true")
    p.add_argument("--no-upper", action="store_true")
    p.add_argument("--no-digits", action="store_true")
    p.add_argument("--no-symbols", action="store_true")
    p.add_argument("--no-ambiguous", action="store_true")
    p.add_argument("-c", "--count", type=int, default=1, help="how many passwords
to generate")
    return p

def run(argv=None):
    args = build_parser().parse_args(argv)
    for _ in range(args.count):
        pw = generate(
            length=args.length,
            lower=not args.no_lower,
            upper=not args.no_upper,
            digits=not args.no_digits,
            symbols=not args.no_symbols,
            no_ambiguous=args.no_ambiguous,
        )
        print(pw)
```

passgen/__main__.py:

```
from .cli import run

if __name__ == "__main__":
    run()
```

passgen/__init__.py:

```
"""Tiny password generator."""
from .core import generate
__version__ = "0.1.0"
```

Expected Interaction

```
$ python -m passgen -l 12
H8q!aKw3#mPx

$ python -m passgen -c 3 --no-symbols -l 8
abcd1234
xy7Az3Qm
nP9k0bWr
```

Extension Challenges

- Add a --avoid CHARS option to exclude specific characters.
- Add a --memorable mode that generates 4 random dictionary words.
- Use secrets instead of random (cryptographically secure).

Grading Rubric

Criterion	Weight
Package layout works	25%
python -m passgen	20%
All CLI flags work	25%
Code organization	15%
Style + docstrings	15%